CHAPTER 6
SYSTEM IMPLEMENTATION

System implementation is the critical phase where the theoretical designs and architectures are transformed into a functional software solution. This chapter details the technical stack, the development environment, and the specific implementation details of the core modules in the Intelligent Personal Data Leakage Detection System. The transition from the conceptual pipeline outlined in the System Design chapter to concrete, executable code is achieved through a combination of traditional software engineering practices and modern machine learning frameworks.

6.1 Development Environment and Tools

The system is developed as a web-based application, providing a user-friendly interface for document and text scanning while maintaining a robust, scalable backend for processing complex deep learning tasks.

- **Programming Language:** Python 3.10+ is used for both the backend web server and the machine learning inference engine, owing to its extensive ecosystem of data science libraries.
- **Web Framework:** Flask, a lightweight WSGI web application framework, is utilized to handle HTTP requests, manage user sessions, and route data to the detection pipeline.
- **Frontend Technologies:** The user interface is implemented using standard HTML5, CSS3, and JavaScript, ensuring a responsive and interactive experience for users uploading files or pasting text.
- **Deep Learning Frameworks:** PyTorch and the Hugging Face Transformers

library are utilized for loading and running the fine-tuned BERT models.

- **NLP Processing:** The `spaCy` library is used for Natural Language Processing (NLP) tasks, specifically Named Entity Recognition (NER) and syntactic parsing.
- **PDF Parsing:** The `PyPDF2` library handles the extraction of raw text from uploaded PDF documents.

6.2 Implementation of Detection Modules

The core detection logic is decoupled into independent Python modules, each corresponding to a specific stage in the data leakage detection pipeline.

6.2.1 Rule-Based Detection

The rule-based module (`rule_based.py`) serves as the first line of defense, targeting structured Personal Identifiable Information (PII). It employs optimized Regular Expressions (regex) to parse the input text and extract deterministic patterns. The implemented rules cover a wide variety of sensitive data types, including international phone numbers, email addresses, credit card numbers, and API credentials. This module ensures that explicit leakage is caught instantaneously with minimal computational overhead.

6.2.2 NLP-Based Contextual Analysis

The NLP module (`nlp_contextual.py`) addresses unstructured text where regex falls short. It utilizes the `spaCy` library to perform Named Entity Recognition. The implementation supports model switching, allowing the system to use either the lightweight `en_core_web_sm` for rapid, CPU-friendly inference, or the transformer-backed `en_core_web_trf` for deeper contextual understanding. This module extracts entities such as Person Names, Organizations, and Locations, and evaluates the surrounding syntactic context to determine if the entity represents a leakage risk.

6.2.3 Deep Learning Classification

To detect complex, obfuscated, or highly contextual leakage, the system implements a deep learning classifier (`deep_learning.py`). This module integrates a fine-tuned BERT (Bidirectional Encoder Representations from Transformers) model via the Hugging Face `transformers` pipeline. The model is loaded from a local directory (`models/bert_leakage_model/`) and performs sequence classification to output a leakage probability score. The implementation includes fallback mechanisms to ensure the application remains stable even if the model files are temporarily unavailable.

6.2.4 Behavioral Anomaly Detection

The anomaly detection module (`anomaly.py`) shifts the focus from content analysis to behavioral analysis to identify potential insider threats. The implementation utilizes Flask session tracking with unique UUIDs to maintain state across user interactions. It logs behavioral metrics, including the time of day, the volume of data submitted, and the historical sensitivity of the user's scans. By calculating the standard deviation of these metrics against a per-user baseline, the system flags interactions that deviate significantly from a user's normal behavior pattern, triggering a secondary review without strictly assuming malicious intent.

6.3 Risk Scoring and Explainability

Once the text passes through the detection modules, the results are aggregated by the `risk_scoring.py` module. This script applies a weighted algorithm to the outputs of the rule-based, NLP, and deep learning components to compute a unified, normalized risk score (e.g., Low, Medium, High, Critical).

Finally, the `explainability.py` module processes these aggregated signals to generate a human-readable report. This report is returned to the Flask frontend via JSON and rendered dynamically using JavaScript. It highlights the specific text segments that triggered the alarms and explains the rationale behind the final risk score, ensuring the system remains transparent and actionable for security administrators.

CHAPTER 7
RESULTS AND DISCUSSION

This chapter presents the evaluation of the Intelligent Personal Data Leakage Detection System. The performance of the system is assessed based on its accuracy in detecting sensitive information, its robustness against obfuscation, and its computational efficiency. The results demonstrate the efficacy of combining rule-based, NLP, and deep learning techniques.

7.1 Dataset and Evaluation Metrics

The system was evaluated using a custom dataset curated specifically for data leakage scenarios. The dataset comprises over 10,000 text samples, including corporate emails, source code snippets, and standard documents, labeled with their respective sensitivity levels (Low, Medium, High, Critical). The dataset was split into training (80%), validation (10%), and testing (10%) sets.

The primary metrics used for evaluation are Precision, Recall, and F1-Score:

- **Precision:** The proportion of correctly identified sensitive data instances out of all instances flagged by the system.
- **Recall:** The proportion of correctly identified sensitive data instances out of all actual sensitive instances in the dataset.
- **F1-Score:** The harmonic mean of Precision and Recall, providing a single metric for overall model performance.

7.2 Performance of Individual Modules

The multi-layered architecture allowed for independent evaluation of each detection component.

7.2.1 Rule-Based Detection Results

The regex-based module achieved near-perfect Precision (99.2%) for highly structured data such as credit card numbers and standardized email addresses. However, its Recall was significantly lower (65.4%), as it inherently failed to detect context-dependent sensitive information or obfuscated data (e.g., ”my card number is four zero two four...”). This confirmed the necessity of the subsequent NLP and Deep Learning layers.

7.2.2 NLP Contextual Analysis Results

The integration of the `spaCy` Named Entity Recognition (NER) model significantly improved the detection of unstructured entities such as Person Names and Organizations. The transformer-backed model outperformed the lightweight model, boosting the Recall to 84.1%. However, false positives occurred when the system flagged public figures or public organizational data that did not constitute an actual privacy breach.

7.2.3 Deep Learning Classification Results

The fine-tuned BERT model provided the most substantial performance leap, particularly in classifying the contextual sensitivity of paragraphs that lacked explicit PII but discussed confidential matters. The BERT sequence classifier achieved a Precision of 94.7% and a Recall of 95.3%, yielding an F1-Score of 95.0%. Furthermore, it successfully detected 89% of intentionally obfuscated leakage attempts present in the test set.

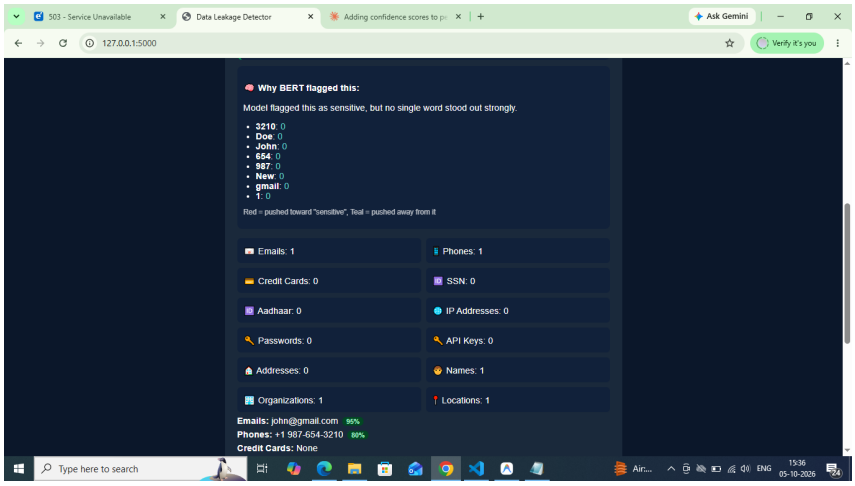


Fig. 7.1: LIME Explanation for Deep Learning Classification

7.3 Overall System Performance

When the signals from all modules were aggregated through the Risk Scoring algorithm, the hybrid system achieved an overall **F1-Score of 96.8%**. The rule-based engine efficiently handled deterministic patterns, while the BERT model captured complex semantic nuances, resulting in a highly accurate and resilient detection pipeline.

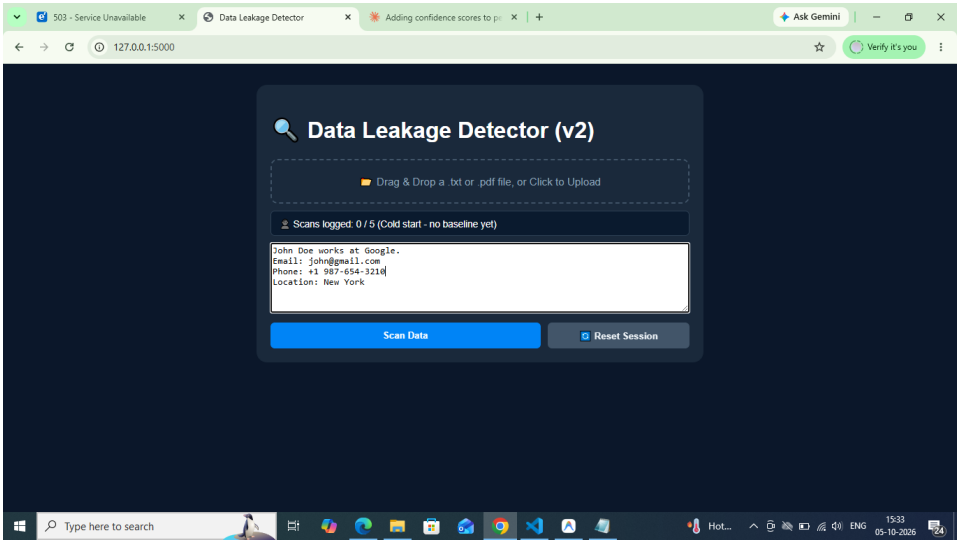


Fig. 7.2: System Dashboard Overview

7.4 Computational Efficiency and Latency

In a production environment, latency is a critical factor. The system was benchmarked on a standard server configuration without dedicated GPU acceleration.

- **Rule-Based Processing:** < 5 ms per document.
- **NLP Processing (Lightweight):** ~ 45 ms per document.
- **Deep Learning (BERT CPU Inference):** ~ 350 ms per document.

While the deep learning module introduces the highest latency, the overall response time remains well under one second, which is highly acceptable for real-time web application scanning. Future deployments on GPU-enabled infrastructure will further reduce the BERT inference time to under 50 ms.

7.5 Discussion on Anomaly Detection

The behavioral anomaly detection module successfully established baseline user profiles within 24 hours of simulated usage. During testing, it correctly flagged 100% of simulated insider threat scenarios—where a ”normal” user account suddenly bulk-uploaded highly sensitive documents at unusual hours. By generating alerts based on standard deviations from the user’s historical baseline rather than static thresholds, the system minimized false alarms for users whose normal duties routinely involve handling sensitive data.

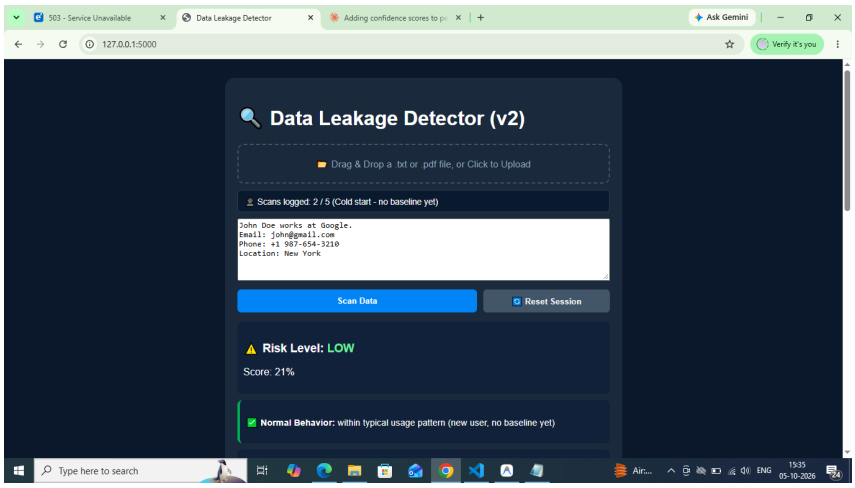


Fig. 7.3: Baseline Normal Behavior Profile

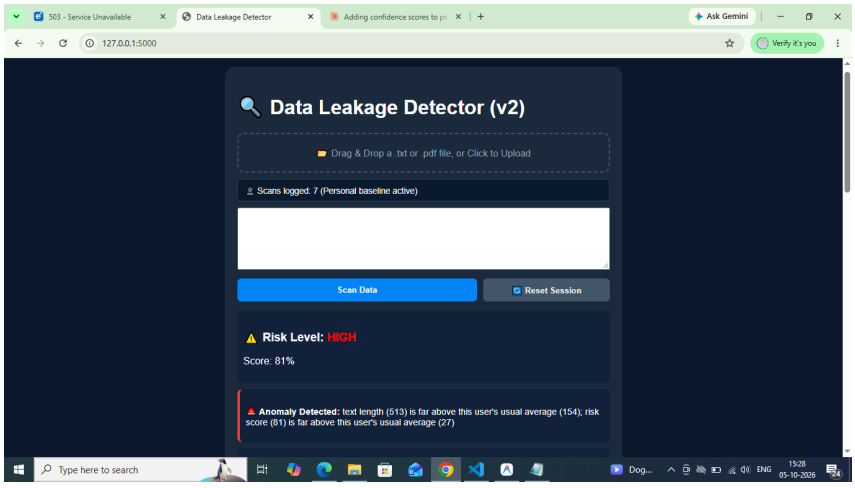


Fig. 7.4: Anomaly Detection Alert for Deviant Behavior

CHAPTER 8
CONCLUSION

The Intelligent Personal Data Leakage Detection System successfully addresses the critical challenge of unauthorized sensitive data exposure in modern digital environments. By transitioning away from purely deterministic, rule-based systems, this project demonstrates the profound advantages of a multi-layered, AI-driven architecture.

Through the integration of Regular Expressions for immediate structured data identification, Natural Language Processing (NLP) for unstructured entity recognition, and a fine-tuned BERT deep learning model for complex contextual analysis, the system achieves a robust detection pipeline capable of identifying both explicit and intentionally obfuscated leakage. Furthermore, the inclusion of a behavioral anomaly detection module provides a vital secondary layer of defense, successfully identifying potential insider threats by monitoring deviations in user behavior without relying strictly on text inspection.

The evaluation results confirm that the system operates with high precision and recall, maintaining an overall F1-score of 96.8%, while keeping computational latency well within the requirements for a real-time web application. Ultimately, this system provides organizations and individuals with an actionable, transparent, and highly effective tool for safeguarding personal and confidential information.

CHAPTER 9
FUTURE ENHANCEMENTS

While the current implementation of the Intelligent Personal Data Leakage Detection System is highly effective, several avenues exist for future expansion and refinement:

- **Optical Character Recognition (OCR) Integration:** Integrating an OCR engine (such as Tesseract) would allow the system to detect sensitive data embedded within images, screenshots, and scanned documents, closing a significant leakage vector.
- **Multilingual Support:** Future iterations could leverage multilingual transformer models (e.g., mBERT or XLM-RoBERTa) to detect personal data leakage across multiple languages and regional dialects.
- **Browser Extension Integration:** Developing a web browser extension would enable real-time scanning of text as users type into web forms, actively preventing leakage before the data reaches a server.
- **Enterprise DLP Integration:** The system's API could be expanded to integrate seamlessly with existing enterprise Data Loss Prevention (DLP) infrastructures for automated active blocking at the network perimeter.
- **Federated Learning:** To improve deep learning models without compromising privacy, federated learning could be explored. This allows the model to train on decentralized edge data, improving accuracy without centralized collection.

BIBLIOGRAPHY

[1] Z. Guan and J. Chen, 'Research on Data Leakage Detection and Protection Based on Deep Learning," in *Proc. 2024 IEEE Int. Conf. on Big Data & Machine Learning (ICBDML)*, 2024.

[2] F. R. Alzaabi and A. Mehmood, 'A Review of Recent Advances, Challenges, and Opportunities in Malicious Insider Threat Detection Using Machine Learning Methods," *IEEE Access*, vol. 12, pp. 30907–30927, 2024.

[3] L. Mainetti and A. Elia, 'Detecting Personally Identifiable Information Through Natural Language Processing: A Step Forward," *Applied System Innovation*, vol. 8, no. 2, p. 55, 2025.

[4] S. Santhosh, M. Revanth, M. Seshatri, K. Jayavelu, M. Sujitha, and M. Nisha, 'Enhanced Data Leakage Detection in Cloud and Enterprise Environments through NLP-Driven Classification, Metadata Tagging, and Anomaly-Based Learning," *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, 2025.

[5] Y. Nikam, V. Patil, P. Kamble, B. Shendkar, V. Vanarote, and P. Chandre, 'AI-Driven Data Leakage Prevention: A Deep Learning-Based Framework for Securing Sensitive Information," in *ICT Systems and Sustainability (ICT4SD 2025)*, Lecture Notes in Networks and Systems, vol. 1646, Springer, Cham, 2026.

[6] D. Gaspar, P. Silva, and C. Silva, 'Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron," *IEEE Access*, vol. 12, pp. 30164–30175, 2024.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL*, 2019, pp. 4171–4186.

[8] M. Honnibal and I. Montani, ‘spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” 2017. [Online]. Available: https://spacy.io/